# Instructor Solution Guide & Master Report: Lab 2 — Network Delay, Packet Loss, and Throughput

**Course:** CS 457: Computer Networks & Socket Programming
**Lab Title:** Lab 2 — Network Delay, Packet Loss, and Throughput with CML and Wireshark
**Document Type:** Instructor Master Copy & Solution Template
**Instructor / Author:** `[ Insert Instructor / TA Name ]`
**Date:** `[ Insert Date ]`
**Semester:** `[ Insert Term, e.g., Fall 2026 ]`

---

## AI Policy & Acceptable Use Guidelines

 AI Policy & Acceptable Use Guidelines - Yellow Icon

Artificial Intelligence tools can be powerful learning aids, but they must be used ethically and responsibly to ensure you develop core networking skills.

- **Lab Assignments:** Do not use AI to perform lab steps or write your lab submissions. However, if you get stuck on a concept or technical obstacle, you may use AI as a virtual tutor to ask clarifying questions and help you troubleshoot.

---

## Executive Summary & Instructor Notes

This document serves as the master instructor copy and solution reference for **Lab 2: Network Delay, Packet Loss, and Throughput**. It contains the complete lab write-up structure, data collection tables, screenshot placeholders with exact figure captions and required visual annotations, and solution answer keys for all theoretical and Wireshark analysis questions.

### Key Pedagogical Objectives

1. **Delay (Latency)**: Demonstrate that Round-Trip Time (RTT) is the sum of directional propagation/transmission delays ($RTT \approx 2 \times \text{One-Way Latency}$).
2. **Packet Loss**: Observe transport-layer recovery mechanisms in Wireshark under 10% link loss (Duplicate ACKs, Out-of-Order segments, Fast Retransmit, Retransmission Timeouts).
3. **Throughput**: Understand the relationship between bottleneck link bandwidth and TCP payload delivery rate, accounting for protocol encapsulation overhead ($\text{Efficiency} \approx 94\% - 97\%$).

---

## 1. CML Topology & Environment Verification

### 1.1 CML Topology Screenshot

Place the annotated screenshot of the Cisco Modeling Labs (CML) canvas below. Ensure that node labels follow the `<initials>-H1` (Client) and `<initials>-H2` (Server) naming format and that the CML Link Condition badge is clearly indicated on the link connecting both hosts.

 Figure 1: CML Topology with Configured Client and Server Nodes and Active Link Condition Badge

*Figure 1: CML workspace topology showing Client node ( `<initials>-H1` ) and Server node ( `<initials>-H2` ) connected via `eth0` with active CML Link Condition impairments applied.*

**Required Visual Annotations on Figure 1**:

- [ ] Bounding box or arrow pointing to Client node `<initials>-H1` ( `192.168.1.1/24` ).
- [ ] Bounding box or arrow pointing to Server node `<initials>-H2` ( `192.168.1.2/24` ).
- [ ] Callout arrow pointing to the **CML Link Condition impairment badge** on the connecting link.

---

### 1.2 Addressing & Initial Connectivity Check

| Device Name | Role | Interface | Assigned IP Address | Subnet Mask | Default Gateway | Baseline Ping RTT (< 1 ms) |
|---|---|---|---|---|---|---|
| `<initials>-H1` | Client | `eth0` | `192.168.1.1` | `255.255.255.0` (`/24`) | N/A | [ ] Pass (`< 0.5 ms`) |
| `<initials>-H2` | Server | `eth0` | `192.168.1.2` | `255.255.255.0` (`/24`) | N/A | [ ] Pass (`< 0.5 ms`) |

---

## 2. Part A: Measuring Link Delay (Latency)

> *[!IMPORTANT]* ***REMINDER: All screenshots, console outputs, and packet captures (.pcap files) must be taken from the SAME active lab session. Mixing screenshots or trace files across different runs or sessions is not permitted.***

### 2.1 Latency Measurement Summary Table

| Test Case | Configured One-Way Latency in CML | Theoretical Round-Trip Time ($2 \times \text{Latency}$) | Observed Min RTT | Observed Avg RTT | Observed Max RTT | Jitter / MDev |
|---|---|---|---|---|---|---|
| **Baseline** | `0 ms` | `< 1.0 ms` | `[ ms ]` | `[ ms ]` | `[ ms ]` | `[ ms ]` |
| **Test 1** | `50 ms` | **100.0 ms** | `[ ms ]` | `[ ms ]` | `[ ms ]` | `[ ms ]` |
| **Test 2** | `150 ms` | **300.0 ms** | `[ ms ]` | `[ ms ]` | `[ ms ]` | `[ ms ]` |

---

### 2.2 Console Output Screenshots for Part A

**Console Ping Summaries (0ms, 50ms, 150ms Latencies)**

 Figure 2: Console Output of Ping RTT Measurements under 0ms, 50ms, and 150ms Latency Settings

*Figure 2: Composite console output from Client H1 executing `ping -c 10 192.168.1.2` under baseline (<1ms), 50ms (average ~100ms), and 150ms (average ~300ms) link conditions.*

**Required Visual Annotations on Figure 2**:

- [ ] Highlight/arrow on baseline ping summary showing `rtt min/avg/max < 1.0 ms` .
- [ ] Highlight/arrow on 50ms latency ping summary showing `avg ~ 100 ms` .

- [ ] Highlight/arrow on 150ms latency ping summary showing `avg ~ 300 ms`.

---

### 2.3 Analysis Answers & Solutions: Section 5.1 (Latency and RTT)

**Question 1: Latency vs. RTT Relationship**

*What is the relationship between the configured one-way link latency in CML and the actual Round-Trip Time (RTT) observed in your ping results? Why does this relationship exist?*

> ***Instructor Solution / Key Concept**:*
>
> - ***Relationship**: $\text{RTT} \approx 2 \times \text{Configured One-Way Link Latency}$.*
> - ***Technical Explanation**: CML applies the configured latency independently to each directional transit across the link. When an ICMP Echo Request is sent from H1 to H2, it experiences the configured delay $d$. When H2 generates and transmits the ICMP Echo Reply back to H1, it traverses the link in the reverse direction and incurs the same delay $d$. Therefore, the total round-trip time recorded by the client ping application is: $$\text{RTT} = d_{\text{outbound}} + d_{\text{inbound}} + d_{\text{processing}} \approx 2d$$*
> - *For a 50ms link condition, $\text{RTT} \approx 50\text{ ms} + 50\text{ ms} = 100\text{ ms}$. For 150ms, $\text{RTT} \approx 150\text{ ms} + 150\text{ ms} = 300\text{ ms}$.*

---

**Question 2: Jitter and RTT Variation in Virtual Environments**

*In Part A, did the RTT remain perfectly constant, or was there variation (jitter)? Explain what factors could introduce RTT variation in a virtual/emulated environment.*

> ***Instructor Solution / Key Concept**:*
>
> - ***Observation**: The RTT is not perfectly constant; there is small measurable variance (jitter, reflected in `mdev` of ~0.2 ms to 1.5 ms).*
> - ***Sources of Variation**:*
>   1. ***Hypervisor / Host OS Scheduling**: Virtual nodes (Alpine Linux) and CML virtual network bridges share host CPU cores. Thread scheduling delays and context switching on the host hypervisor introduce non-deterministic microsecond-to-millisecond delays.*
>   2. ***Software-Defined Emulation Queuing**: CML's Linux Traffic Control ( `tc` / `netem` ) subsystem buffers packets in software queues to simulate latency, introducing minor processing jitter.*
>   3. ***Guest Kernel Interrupt Handling & Timer Resolution**: Linux kernel tick rate, socket buffer allocation, and ICMP checksum validation add small processing delays ($d_{\text{proc}}$).*

---

# 3. Part B: Measuring Packet Loss & TCP Analysis

## 3.1 Packet Loss Data Summary Table

Link Configuration: **Packet Loss = 10%**, **Latency = 0 ms**, **Bandwidth = Full / Unconstrained**.

| Metric | Measured Value | Expected Range | Notes |
|---|---|---|---|
| **Packets Transmitted** | `50` | `50` | Sent via `ping -c 50 192.168.1.2` |
| **Packets Received** | `[ Insert ]` | `42 - 48` | |

| | | | |
|---|---|---|---|
| **Packet Loss Percentage** | `[ Insert % ]` | `4.0% - 16.0%` | Statistical variation around nominal 10% rate |
| **TCP Download Behavior (`wget`)** | `[ Insert ]` | Stuttering / pauses | Transfer pauses during packet drops and retransmissions |

---

## 3.2 Console Output Screenshot for Part B

**50-Packet Ping Test Output (10% Loss)**

 Figure 3: Console Output of 50-Packet Ping Test Demonstrating Packet Loss

*Figure 3: Console output from Client H1 showing packet loss statistics (~10% packet drop) after executing `ping -c 50 192.168.1.2`.*

**Required Visual Annotations on Figure 3**:

- ☐ Bounding box around packet loss summary line ( `--- 192.168.1.2 ping statistics ---` ).
- ☐ Callout arrow pointing to the percentage packet loss value (e.g., `8% packet loss` or `10% packet loss` ).

---

## 3.3 Wireshark Loss Analysis Screenshots (Part B)

**Missing ICMP Request Gap**

 Figure 4: Wireshark Packet List Filtered on ICMP Showing Missing Sequence Number Gap

*Figure 4: Wireshark capture ( `packet_loss.pcap` ) filtered on `icmp` showing a missing Echo Request sequence number where a packet was dropped by the impaired link.*

**Required Visual Annotations on Figure 4**:

- ☐ Arrow pointing to the last received packet before the gap ( `icmp.seq = N` ).
- ☐ Arrow pointing to the subsequent packet after the gap ( `icmp.seq = N + 1` or `N + 2` ).
- ☐ Text callout noting the missing sequence number dropped by CML link condition.

---

**TCP Loss Recovery Flags (Retransmission, Duplicate ACK, Out-of-Order)**

 Figure 5: Wireshark Packet List Filtered on TCP Analysis Flags Showing Loss Recovery Mechanisms

*Figure 5: Wireshark trace filtered on `tcp.analysis.flags` clearly displaying `[TCP Retransmission]`, `[TCP Dup ACK]`, and `[TCP Out-of-Order]` indicators.*

**Required Visual Annotations on Figure 5**:

- ☐ Red highlight/arrow on a **`[TCP Retransmission]`** or **`[TCP Fast Retransmission]`** frame.
- ☐ Yellow highlight/arrow on a **`[TCP Dup ACK]`** frame showing receiver acknowledging the last contiguous byte.
- ☐ Blue highlight/arrow on a **`[TCP Out-of-Order]`** frame arriving ahead of the missing dropped segment.

---

### 3.4 Analysis Answers & Solutions: Section 5.2 (Packet Loss)

**Question 3: ICMP Sequence Gap Identification**

*Filter `packet_loss.pcap` in Wireshark for `icmp`. Look at the sequence numbers of the ICMP Echo Requests (`icmp.seq`). Do you see gaps in the sequence numbers? Find one missing request and provide an annotated screenshot pointing out the packet numbers immediately before and after the gap.*

> ***Instructor Solution / Key Concept**:*
>
> - ***Identification**: In Wireshark with filter `icmp`, examining the `icmp.seq` (or `icmp.seq_le`) column reveals that sequence numbers increment sequentially: 1, 2, 3, 4, 6, 7... (e.g., Sequence 5 is missing).*
> - ***Annotated Figure 4**: Points out Frame $A$ (`icmp.seq = 4`) and Frame $B$ (`icmp.seq = 6`), confirming packet 5 was dropped in transit by the link emulator.*

---

**Question 4: Server Reply Behavior on Lost Requests**

*When an ICMP Echo Request is lost due to packet loss, does the server send an Echo Reply? Explain using your Wireshark observations.*

> ***Instructor Solution / Key Concept**:*
>
> - ***Answer**: No, the server does not send an Echo Reply.*
> - ***Technical Explanation**: ICMP is a connectionless, stateless protocol built directly on IP with no built-in transport-layer retransmission or acknowledgment timer at the server. If the Echo Request packet is dropped by the link before reaching the server, the server's network stack never receives the frame and has no knowledge that a request was transmitted. Therefore, no Echo Reply is ever generated.*

---

**Question 5: Technical Explanation of TCP Analysis Flags**

*Locate examples of TCP Retransmissions, TCP Duplicate ACKs, and TCP Out-of-Order Segments in Wireshark. Clearly explain what each flag represents:*

> ***Instructor Solution / Key Concept**:*
>
> 1. ***TCP Retransmission (`[TCP Retransmission]` / `[TCP Fast Retransmission]`)**:*
>    - *Occurs when the sender re-sends a segment whose sequence number was already transmitted earlier. Triggered either by the expiration of the Retransmission Timeout (RTO) timer or by the receipt of 3 duplicate ACKs (Fast Retransmit).*
> 2. ***TCP Duplicate ACK (`[TCP Dup ACK]`)**:*
>    - *Occurs when the receiving host receives an out-of-order segment (a sequence number higher than expected). Because TCP uses cumulative acknowledgments, the receiver cannot acknowledge the new data; instead, it immediately re-sends an ACK for the last contiguous byte received to prompt the sender to retransmit the missing byte.*
> 3. ***TCP Out-of-Order Segment (`[TCP Out-of-Order]`)**:*
>    - *Occurs when a segment arrives with a sequence number higher than the next expected sequence number, and subsequent packets arrive while an earlier segment was dropped.*

---

**Question 6: Out-of-Order Segments, Duplicate ACKs, and Fast Retransmit**

*Explain the technical relationship between out-of-order segments, duplicate ACKs, and TCP Fast Retransmit:*

- *Why does an out-of-order segment cause the receiving TCP stack to immediately generate a duplicate ACK?*

- *How many duplicate ACKs must the sender receive before triggering a Fast Retransmit, and why does this prevent waiting for the Retransmission Timeout (RTO) timer to expire?*

> ***Instructor Solution / Key Concept**:*
>
> - ***Immediate Dup ACK Generation**: When a segment is lost, any subsequent segment arriving at the receiver creates a "gap" in the byte stream. To alert the sender without delaying, RFC 5681 mandates that the receiver must immediately generate a duplicate ACK acknowledging the highest contiguous in-order byte.*
> - ***Threshold**: The sender requires **3 Duplicate ACKs** (4 identical ACKs in total) to trigger **Fast Retransmit**.*
> - ***Why 3 Dup ACKs?**: A single or double duplicate ACK can be caused merely by temporary network packet reordering. Receiving 3 consecutive duplicate ACKs provides strong statistical confidence that a packet was actually lost rather than reordered. Fast Retransmit immediately retransmits the missing segment without waiting for the slow RTO timer (which can take 200ms to 1000ms+), significantly improving throughput.*

---

**Question 7: Impact of Packet Loss on Bulk TCP Transfer Duration**

*How does packet loss affect the overall transmission time of a TCP stream compared to a lossless link? Refer to your `wget` results.*

> ***Instructor Solution / Key Concept**:*
>
> - ***Impact**: Packet loss causes a drastic increase in total download duration.*
> - ***Technical Explanation**: When TCP detects packet loss, its Congestion Control algorithm (AIMD) cuts the Congestion Window ( `cwnd` ) in half (or collapses it to 1 MSS on timeout) and exits Congestion Avoidance. In addition to the sender stalling during retransmissions, the reduced window throttles the transmission rate, forcing TCP to slowly probe bandwidth again in additive increase. On an impaired 10% loss link, `wget` exhibits visible stalls and takes several times longer to transfer the 10MB test file.*

---

# 4. Part C: Measuring Throughput (Bandwidth Constraints)

## 4.1 Throughput & Bandwidth Comparison Table

File Downloaded: `http://192.168.1.2/large.bin` (Size: **10,485,760 Bytes** / **10.0 MB**).

| Configured Bandwidth Limit | Observed `wget` Transfer Rate | Calculated Throughput ($\text{Rate} \times 8$) | Theoretical Max Capacity | Protocol Efficiency Ratio ($\frac{\text{Calculated}}{\text{Configured}} \times 100\%$) |
|---|---|---|---|---|
| **10 Mbps** ($10,000\text{ kbps}$) | `[ ~1.18 MB/s ]` | `[ ~9.44 Mbps ]` | **10.00 Mbps** | `[ ~94.4% ]` |
| **500 kbps** ($500,000\text{ bps}$) | `[ ~59.2 KB/s ]` | `[ ~473.6 kbps ]` | **500.00 kbps** | `[ ~94.7% ]` |

---

## 4.2 Console Output Screenshots for Part C

**`wget` Download Output at 10 Mbps and 500 kbps Limits**

 Figure 6: Console Output of wget Downloads Under 10 Mbps and 500 kbps Bandwidth Limits

*Figure 6: Console output from Client H1 executing `wget -0 /dev/null http://192.168.1.2/large.bin` showing transfer speeds stabilized at ~1.2 MB/s (10 Mbps limit) and ~60 KB/s (500 kbps limit).*

**Required Visual Annotations on Figure 6**:

- [ ] Highlight/callout on 10 Mbps `wget` result showing transfer speed in `MB/s`.
- [ ] Highlight/callout on 500 kbps `wget` result showing transfer speed in `KB/s`.

---

## 4.3 Wireshark Throughput Graph Screenshot (Part C)

**Wireshark I/O Graph / TCP Stream Throughput Graph**

 Figure 7: Wireshark I/O Graph of TCP Throughput Showing Bandwidth Limiting Plateaus

*Figure 7: Wireshark I/O Graph ( `Statistics -> I/O Graphs` with Y-axis = `Bits/s` for `tcp.stream == 0` ) showing the steady-state throughput plateau constrained by CML's link shaper.*

**Required Visual Annotations on Figure 7**:

- [ ] Arrow/callout pointing to the horizontal plateau region at 10 Mbps or 500 kbps.
- [ ] Callout confirming the Y-axis units ( `Bits/s` or `Bytes/s` ).

---

## 4.4 Analysis Answers & Solutions: Section 5.3 (Throughput and Bandwidth)

**Question 8: Wireshark Throughput Graph Interpretation**

*Provide an annotated screenshot of your Wireshark I/O Graph or Throughput Graph showing the TCP download stream and pointing out the plateau regions.*

> ***Instructor Solution / Key Concept**:*
>
> - ***Graph Analysis**: In Figure 7, the graph trace rises rapidly during TCP slow start and immediately flattens out into a steady horizontal plateau matching the configured link limit.*
> - ***Why it Plateaus**: CML's link condition enforces a token-bucket / traffic-shaping rate limiter. When the TCP sender attempts to send faster than the configured bandwidth, excess packets are queued or delayed, throttling the rate of ACKs returned and locking the steady-state transfer rate to the link ceiling.*

---

**Question 9: Bandwidth to Throughput Conversion & Efficiency Analysis**

*Compare your download speeds from Part C. When the bandwidth limit was 10 Mbps, what was the actual download speed reported by `wget` in MB/s? When the limit was 500 kbps, what was the speed in KB/s? Convert your download speeds to bits per second and calculate the efficiency ratio.*

> ***Instructor Solution / Key Concept**:*
>
> - ***10 Mbps Calculation**: $$\text{Observed Rate} \approx 1.18\text{ MB/s} = 1.18 \times 10^6\text{ Bytes/sec}$$ $$\text{Throughput in bps} = 1.18 \times 10^6 \times 8 = 9.44\text{ Mbps}$$ $$\text{Efficiency} = \frac{9.44\text{ Mbps}}{10.0\text{ Mbps}} \times 100% \approx 94.4%$$*

> - ***500 kbps Calculation**: $$\text{Observed Rate} \approx 59.2\text{ KB/s} = 59,200\text{ Bytes/sec}$$ $$\text{Throughput in bps} = 59,200 \times 8 = 473.6\text{ kbps}$$ $$\text{Efficiency} = \frac{473.6\text{ kbps}}{500\text{ kbps}} \times 100% \approx 94.7%$$*

---

**Question 10: Bandwidth vs. Throughput & Encapsulation Overhead**

*Explain the difference between throughput and bandwidth. Which one represents the theoretical maximum capacity of the link, and which one represents the actual observed rate of successful data delivery? Why is measured throughput almost always slightly lower than theoretical link bandwidth (consider packet encapsulation overhead)?*

> ***Instructor Solution / Key Concept**:*
>
> - ***Bandwidth**: The theoretical maximum physical bit-rate capacity of the communication channel (measured in bits per second, bps). It represents the width of the pipe.*
> - ***Throughput (Goodput)**: The actual observed rate of successful application payload data delivered from source to destination per unit time.*
> - ***Why Throughput < Bandwidth (Overhead Sources)**:*
>   1. ***Protocol Header Encapsulation**: Every TCP segment carries:*
>      - *Ethernet Frame Header & Preamble: $14 + 4 + 8 = 26\text{ Bytes}$*
>      - *IPv4 Header: $20\text{ Bytes}$*
>      - *TCP Header: $20 - 32\text{ Bytes}$ (with options) For a standard 1500-byte MTU, payload data is $\approx 1460\text{ Bytes}$. Header overhead consumes $\approx \frac{66}{1500} \approx 4.4%$ of the total transmitted bits.*
>   2. ***TCP Acknowledgment Traffic & Inter-Frame Gaps (IFG)**: Reverse-path ACK processing, TCP window sliding dynamics, and Ethernet Interpacket Gaps consume channel resources.*
>   3. ***Application Protocol Overhead**: HTTP headers ( `Content-Length` , `Content-Type` ) and `wget` buffer processing introduce minor measurement delta.*

---

## 5. Troubleshooting & Common Pitfalls

**Issue Encountered:** [Describe what went wrong, if anything]
**Discovery Method:** [How did you identify the issue?]
**Resolution:** [How did you resolve it?]

> ***Instructor Notes on Common Student Pitfalls**:*
>
> 1. ***Unsaved Link Impairments**: Students frequently forget to click **Save** or **Update** in the CML Link Condition tab, resulting in ping tests continuing to show latency.*
> 2. ***Bandwidth Unit Confusion**: Entering in Mbps instead of leads to unconstrained download speeds during Part C.*
> 3. ***Unannotated Screenshots**: Students often submit raw Wireshark windows without callout arrows identifying missing ICMP sequence numbers or TCP analysis flags.*

---

## 6. Lab Deliverables & Submission Verification

Verify that all required files, figures, and trace captures are attached:

- [ ] **Completed Master Report / Student Submission document**.
- [ ] **Figure 1**: Annotated CML Topology Screenshot ( `images/figure_1_cml_topology.png` ).
- [ ] **Figure 2**: Annotated Console Output for Latency Tests ( `images/figure_2_ping_latency_summary.png` ).

- [ ] **Figure 3**: Annotated Console Output for 10% Packet Loss ( `images/figure_3_ping_packet_loss.png` ).
- [ ] **Figure 4**: Annotated Wireshark ICMP Missing Sequence Gap ( `images/figure_4_wireshark_icmp_loss_gap.png` ).
- [ ] **Figure 5**: Annotated Wireshark TCP Analysis Flags ( `images/figure_5_wireshark_tcp_loss_flags.png` ).
- [ ] **Figure 6**: Annotated Console Output for `wget` Throughput ( `images/figure_6_wget_throughput.png` ).
- [ ] **Figure 7**: Annotated Wireshark I/O Throughput Graph ( `images/figure_7_wireshark_throughput_graph.png` ).
- [ ] **PCAP File 1**: **`packet_loss.pcap`** (Part B trace).
- [ ] **PCAP File 2**: **`throughput_test.pcap`** (Part C trace).